Securing a Shared S3 Bucket — Lab Report

15 October 2026 · Abdo

The bucket anycompany-data-[redacted] in Europe (Ireland) now gives the data team exactly the access in the target model, and only public/ is readable by anonymous visitors, over HTTPS only. 14 of the 18 tests are fully evidenced and matched the expected result; tests 3 and 4 are partly evidenced, and tests 12 and 14 still need to be run or captured.

Phase 1 — Prepare and inspect the bucket (Task 1)

No instructor bucket was provided, so the lab bucket was created with default settings and seeded with the six lab objects. Before any change, nothing in it was public.

Item	Value
Bucket	anycompany-data-[redacted]
Region	Europe (Ireland) eu-west-1
Account ID	[redacted]
Bucket ARN	arn:aws:s3:::anycompany-data-[redacted]

1. Created a General purpose bucket with the defaults: ACLs disabled (bucket owner enforced), Block all public access on, versioning off, SSE-S3 encryption.
2. Created the folders shared/, reports/, restricted/, public/ and home/, then uploaded the seed objects: shared/project-plan.txt, reports/q3-summary.csv, restricted/salaries.csv (fake data), public/index.html and public/logo.png. home/ was left empty.
3. Opened the Object URL of public/index.html in a private window. S3 returned an XML AccessDenied error.
4. Recorded the Permissions tab: all four Block Public Access settings On, no bucket policy, Object Ownership = bucket owner enforced.
5. Checked account-level Block Public Access: Off. Only the bucket-level setting therefore had to change in Phase 4.

Why step 3 is denied: S3 is private by default. No policy grants anonymous s3:GetObject, and Block Public Access would block such a policy anyway.



Figure 1 — The bucket created in eu-west-1.

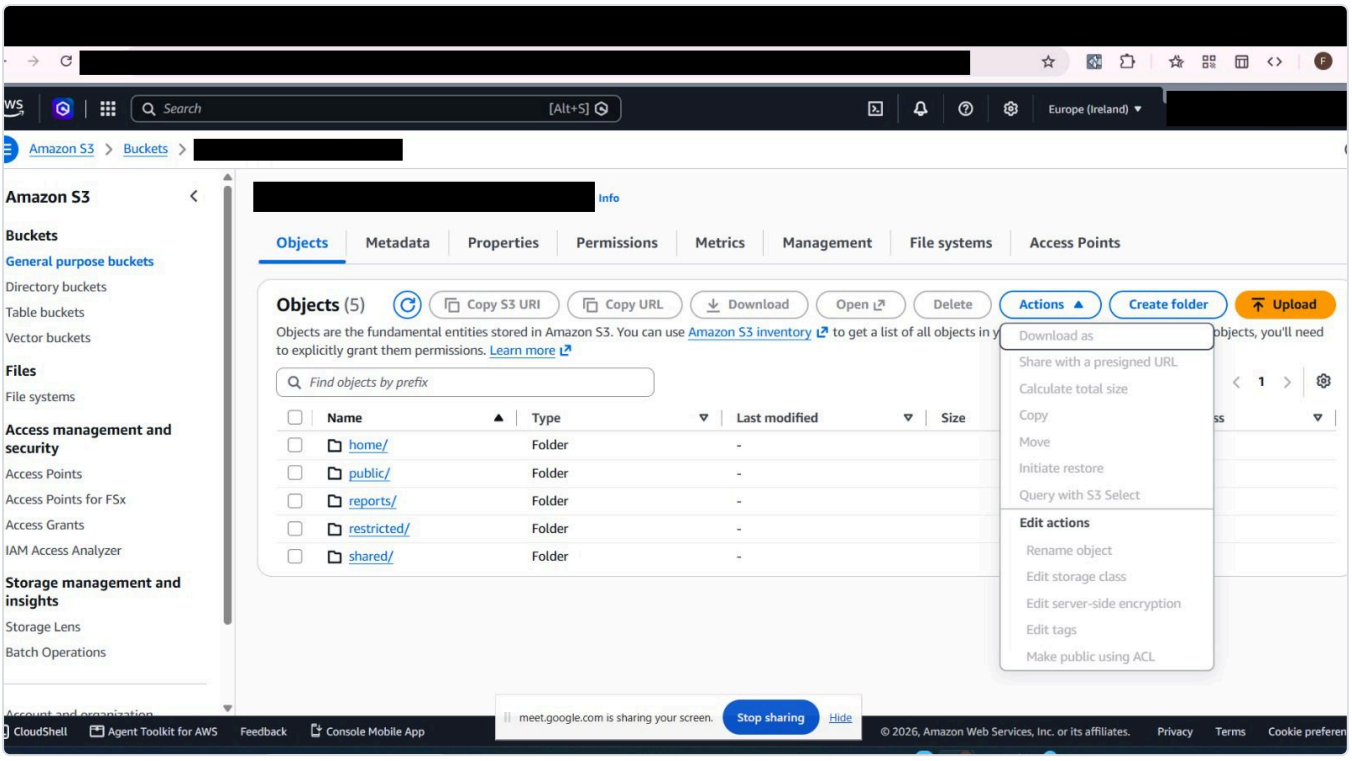


Figure 2 — The five top-level folders, seen as the administrator.



Figure 3 — Step 3: anonymous request to public/index.html returns AccessDenied before any change.

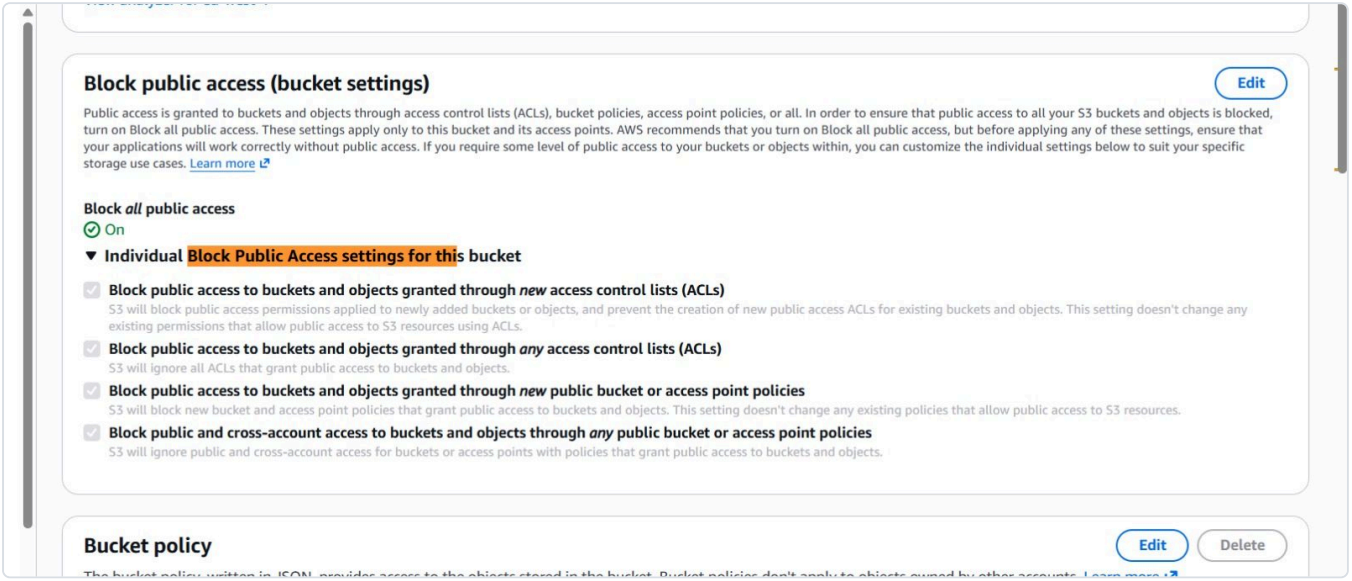


Figure 4 — Step 4: Block all public access On, all four settings ticked.

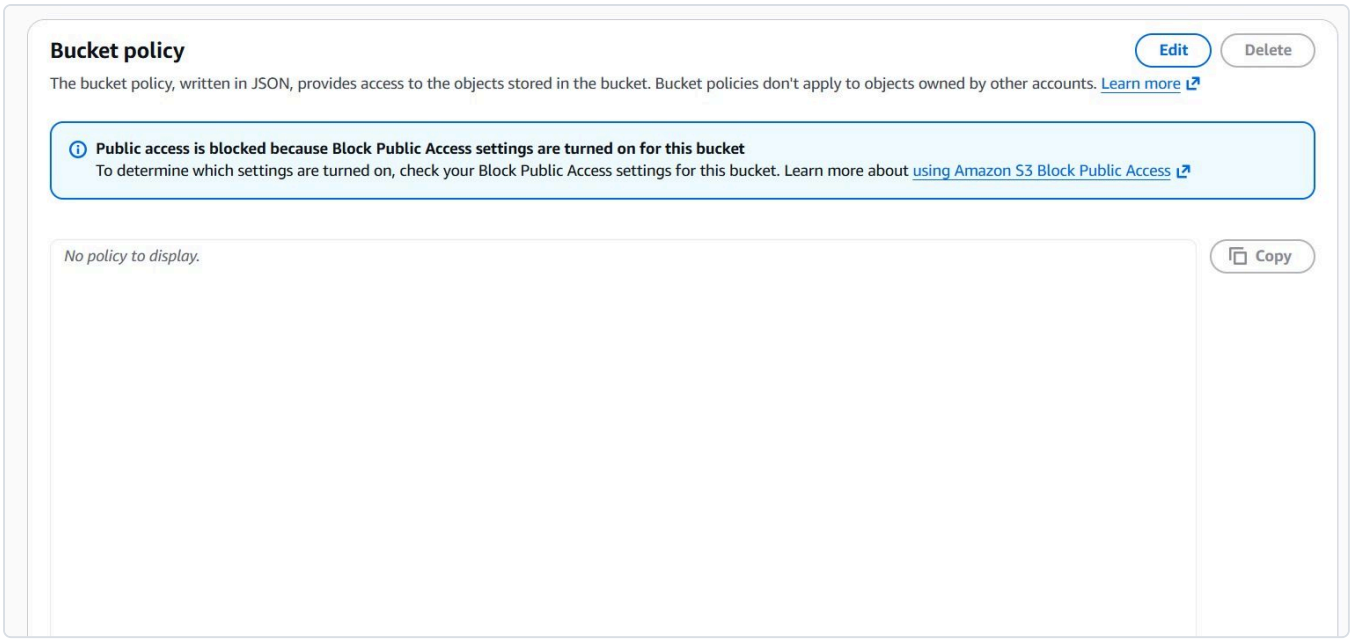


Figure 5 — Step 4: no bucket policy, and public access blocked by Block Public Access.

Phase 2 — IAM groups and users (Task 2)

Two groups and three console-only users were created. Permissions are granted through the groups, never attached to a user, so a new hire only needs a group membership.

User	DataTeam-Members	DataTeam-Leads
team-member-1	Yes	No
team-member-2	Yes	No
team-lead	Yes	Yes

- Groups were created empty first; their policies were attached in Phase 3.
- Each user has console access only, with a custom password and no access keys. "Must create a new password at next sign-in" was cleared to speed up testing.
- team-lead sits in both groups because permissions add up: DataTeam-Members gives the base access, and DataTeam-Leads adds only publishing. Without the Members group, the lead could not even list the bucket.
- Checkpoint: DataTeam-Members shows 3 users and DataTeam-Leads shows 1 user.
- Test sign-in URL: https://[redacted].signin.aws.amazon.com/console

Phase 3 — Team permissions (Task 3)

Two customer managed policies carry all team permissions: DataTeam-S3-Base is attached to DataTeam-Members, and DataTeam-S3-Publish to DataTeam-Leads. restricted/ is closed simply because no statement grants it (implicit deny).

Policy	Statement	Actions	Resource	What it allows
DataTeam-S3-Base	AllowConsoleBucketDiscovery	ListAllMyBuckets, GetBucketLocation	`*`	Shows the bucket list in the console; no object access
DataTeam-S3-Base	ListRootAndHomeFolder	ListBucket, prefix `""` or `home/`, delimiter `/`	Bucket ARN	Top-level folders and the folder names under home/, one level deep
DataTeam-S3-Base	ListTeamPrefixes	ListBucket, prefix `shared/*`, `reports/*`, `public/*`, `home/${aws:username}/*`	Bucket ARN	Listing inside the team folders and the user's own home only
DataTeam-S3-Base	ReadWriteSharedAndOwnHome	GetObject, PutObject, DeleteObject	`shared/*`, `home/${aws:username}/*`	Read and write shared work and the user's own home
DataTeam-S3-Base	ReadReportsAndPublic	GetObject	`reports/*`, `public/*`	Read published content
DataTeam-S3-Publish	PublishReportsAndPublicContent	PutObject, DeleteObject	`reports/*`, `public/*`	The lead publishes and removes reports and public files

- ${aws:username} is replaced at request time with the caller's IAM user name, so one policy gives every user a private home folder.
- s3:ListBucket is granted on the bucket ARN, and object actions on object ARNs (bucket/prefix/*). Swapping them saves without error but grants nothing.
- Checkpoint: team-lead's Permissions tab lists both policies, each marked "Attached via group".

DataTeam-S3-Base:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowConsoleBucketDiscovery",
      "Effect": "Allow",
      "Action": ["s3:ListAllMyBuckets", "s3:GetBucketLocation"],
      "Resource": "*"
    },
    {
      "Sid": "ListRootAndHomeFolder",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::anycompany-data-[redacted]",
      "Condition": {
        "StringEquals": {
          "s3:prefix": ["", "home/"],
          "s3:delimiter": ["/"]
        }
      }
    },
    {
      "Sid": "ListTeamPrefixes",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::anycompany-data-[redacted]",
      "Condition": {
        "StringLike": {
```

```
          "s3:prefix": ["shared/*", "reports/*", "public/*", "home/${aws:username}/*"]
        }
      }
    },
    {
      "Sid": "ReadWriteSharedAndOwnHome",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": [
        "arn:aws:s3:::anycompany-data-[redacted]/shared/*",
        "arn:aws:s3:::anycompany-data-[redacted]/home/${aws:username}/*"
      ]
    },
    {
      "Sid": "ReadReportsAndPublic",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": [
        "arn:aws:s3:::anycompany-data-[redacted]/reports/*",
        "arn:aws:s3:::anycompany-data-[redacted]/public/*"
      ]
    }
  ]
}
```

DataTeam-S3-Publish:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublishReportsAndPublicContent",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:DeleteObject"],
      "Resource": [
        "arn:aws:s3:::anycompany-data-[redacted]/reports/*",
        "arn:aws:s3:::anycompany-data-[redacted]/public/*"
      ]
    }
  ]
}
```

Phase 4 — Make public/ publicly readable (Task 4)

Anonymous users have no IAM identity, so their access can only come from the bucket policy. Block Public Access was relaxed for policies only, then a bucket policy granted anonymous read on public/* and denied every non-HTTPS request.

Block Public Access setting (bucket)	Value	Why
Public access through new ACLs	On	ACLs are never used
Public access through any ACLs	On	ACLs are never used
Public access through new public bucket or access point policies	Off	Lets a public bucket policy be saved
Public and cross-account access through any public bucket or access point policies	Off	Lets that policy take effect

Bucket policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadForPublicPrefixOnly",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::anycompany-data-[redacted]/public/*"
    },
    {
      "Sid": "DenyNonHttpsRequests",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::anycompany-data-[redacted]",
        "arn:aws:s3:::anycompany-data-[redacted]/*"
      ],
      "Condition": {
        "Bool": { "aws:SecureTransport": "false" }
      }
    }
  ]
}
```

- PublicReadForPublicPrefixOnly lets anyone read objects under public/. It grants no s3:ListBucket, so visitors cannot browse the folder and need the exact URL.
- The Resource ends in `/public/*`, not `/*`. A bare `/*` would have published restricted/salaries.csv to the internet.
- DenyNonHttpsRequests is an explicit deny, so it overrides every allow, administrator included, whenever a request arrives over plain HTTP.
- Checkpoint: the bucket now shows as "Publicly accessible", and the same anonymous request that failed in Phase 1 now loads the page.

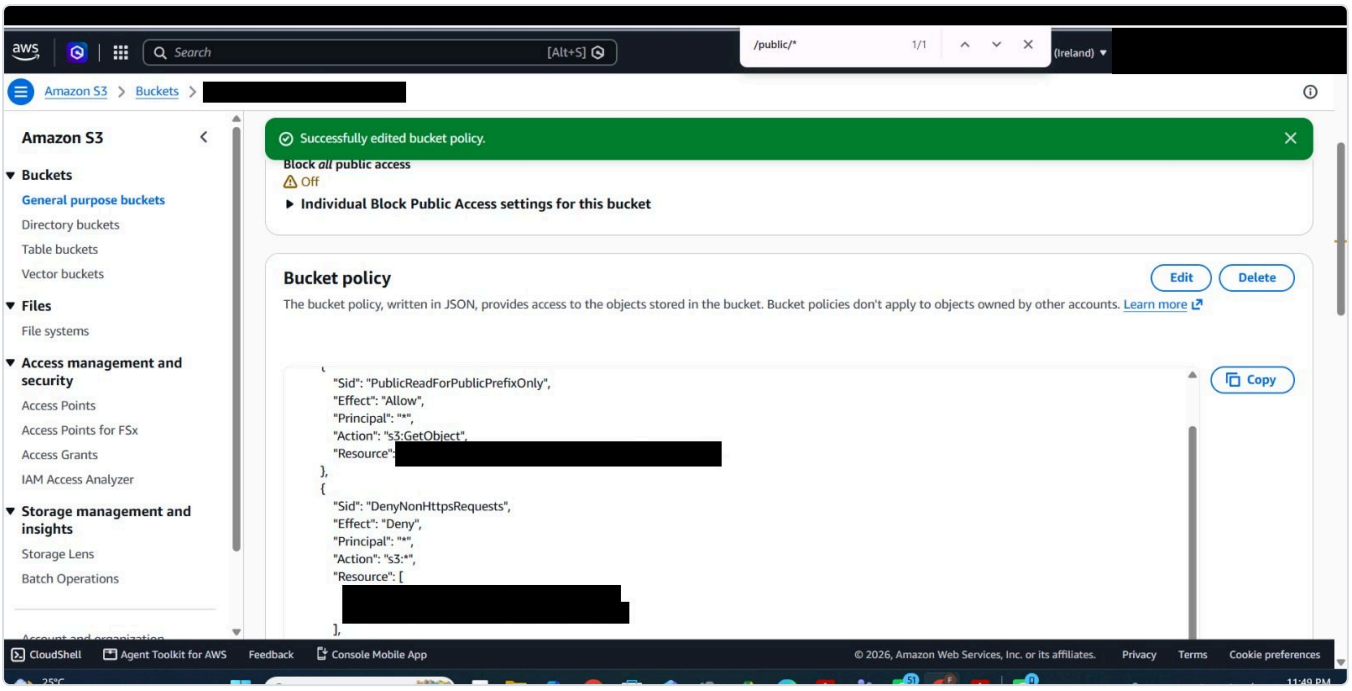


Figure 6 — Bucket policy saved, with the public statement limited to `/public/*`; Block all public access now Off at bucket level.



Figure 7 — Anonymous request to public/index.html now returns the page (the logo image does not load; see Still to capture).

Phase 5 — Verification results (Task 5)

Every test that was captured returned the expected result: each allowed action succeeded and each denied action failed. The required deliverable screenshots are tests 4, 7, 13 and 15 (Figures 11, 14, 21 and 22).

Each persona was tested in a private browser window, signing out between users. Anonymous tests ran in a signed-out window, and test 18 ran with `curl.exe -i` in PowerShell, because browsers often upgrade http:// to https:// silently.

#	Persona	Action	Expected	Result	Status	Evidence
1	team-member-1	Open the bucket and see the top-level folders	Success	Success: five folders listed	**Pass**	Figure 8
2	team-member-1	Download shared/project-plan.txt	Success	Success: file downloaded	**Pass**	Figure 9
3	team-member-1	Upload a file to shared/	Success	Upload screen captured; completion not captured	**Partly evidenced**	Figure 10
4	team-member-1	Create home/team-member-1/ and upload a file into it	Success	Folder created; the upload into it was not captured	**Partly evidenced**	Figure 11
5	team-member-1	Open home/team-member-2/	Access denied	Insufficient permissions to list objects	**Pass**	Figure 12
6	team-member-1	Download reports/q3-summary.csv	Success	Success: file downloaded	**Pass**	Figure 13
7	team-member-1	Upload a file to reports/	Access denied	Upload failed: Access denied	**Pass**	Figure 14
8	team-member-1	Upload a file to public/	Access denied	Upload failed: Access denied	**Pass**	Figure 15
9	team-member-1	Open restricted/	Access denied	Insufficient permissions to list objects	**Pass**	Figure 16
4 (5.2)	team-member-2	Create home/team-member-2/	Success	Folder created	**Pass**	Figure 17
5 (5.2)	team-member-2	Open home/team-member-1/	Access denied	Insufficient permissions to list objects	**Pass**	Figure 18
10	team-lead	Upload a file to reports/	Success	Upload succeeded (a PNG was used instead of q4-summary.csv)	**Pass**	Figure 19
11	team-lead	Upload a new file to public/	Success	Upload succeeded (a PNG was used instead of notice.txt)	**Pass**	Figure 20

#	Persona	Action	Expected	Result	Status	Evidence
12	team-lead	Open restricted/	Access denied	Not captured	To capture	—
13	Anonymous	Open public/index.html over HTTPS	Page loads (HTTP 200)	Page loads	Pass	Figure 21
14	Anonymous	Open the file the lead published in test 11	File loads	Not completed: the notice.txt URL was tested, but the lead had uploaded a different file name	To capture	—
15	Anonymous	Open restricted/salaries.csv	AccessDenied (HTTP 403)	AccessDenied	Pass	Figure 22
16	Anonymous	Open shared/project-plan.txt	AccessDenied (HTTP 403)	AccessDenied	Pass	Figure 23
17	Anonymous	List public/ with ?list-type=2&prefix=public/	AccessDenied (no listing)	AccessDenied	Pass	Figure 24
18	Anonymous	Request public/index.html over plain HTTP	AccessDenied (HTTP 403)	HTTP/1.1 403 Forbidden, AccessDenied	Pass	Figure 25
5.5	Policy Simulator	team-member-1: GetObject, PutObject on restricted/salaries.csv	Both denied	Simulator configured; run result not captured	To capture	Figure 26
5.5	Policy Simulator	team-member-1: GetObject, PutObject on home/team-member-1/notes.txt	Both allowed	Not captured	To capture	—

Test 18 is the HTTPS-only proof: the same object loads over HTTPS in test 13, so the 403 comes from the DenyNonHttpsRequests statement alone.

Phase 5 evidence — screenshots by persona

The screenshots below are in test order and are numbered to match the Evidence column of the results table.

5.1 team-member-1 (tests 1–9)

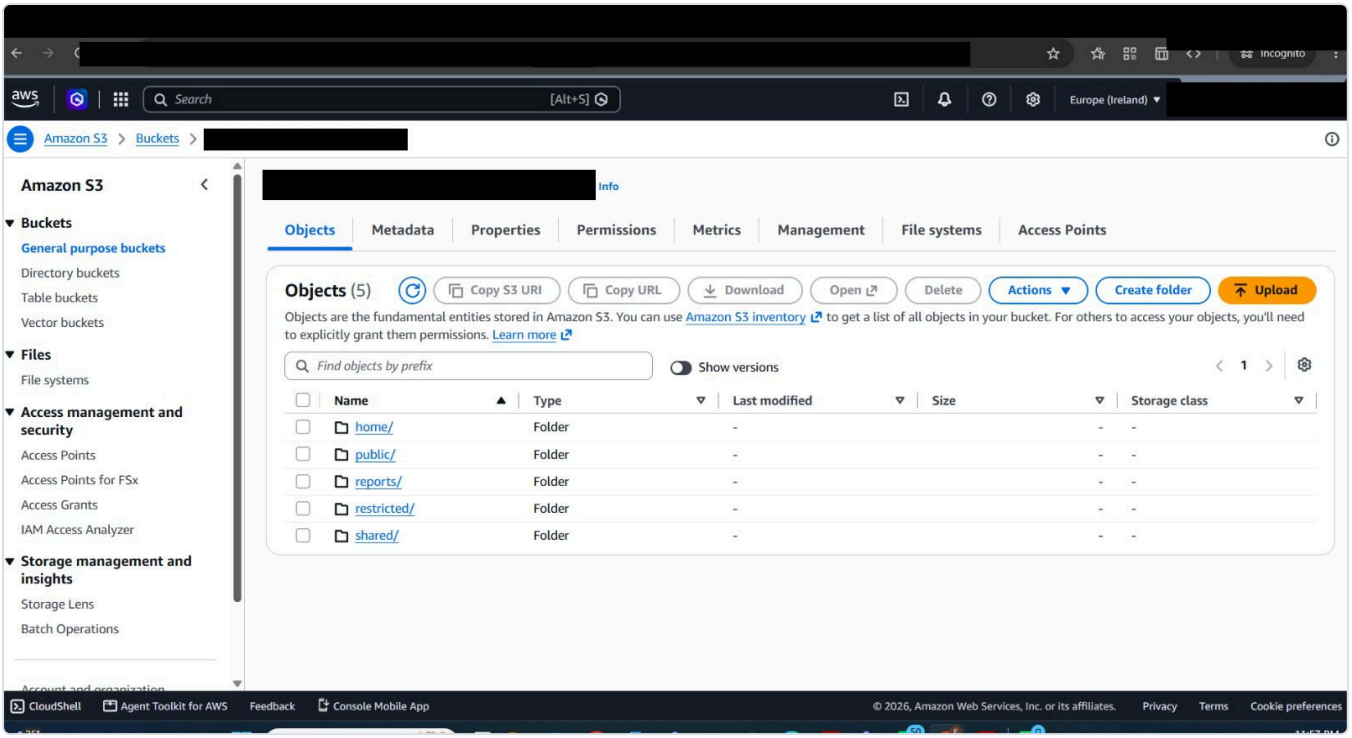


Figure 8 — Test 1: signed in as team-member-1, the bucket opens and lists the five top-level folders.

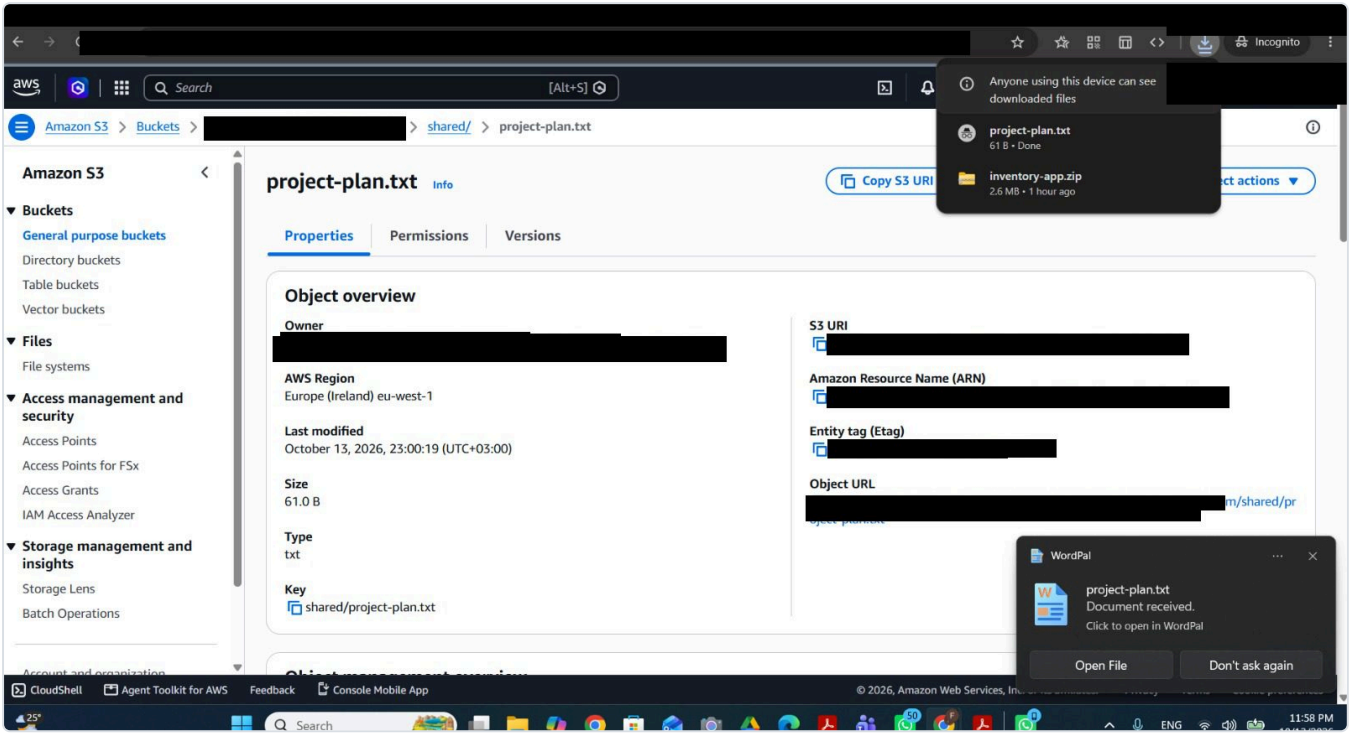


Figure 9 — Test 2: team-member-1 downloads shared/project-plan.txt (download panel, top right).

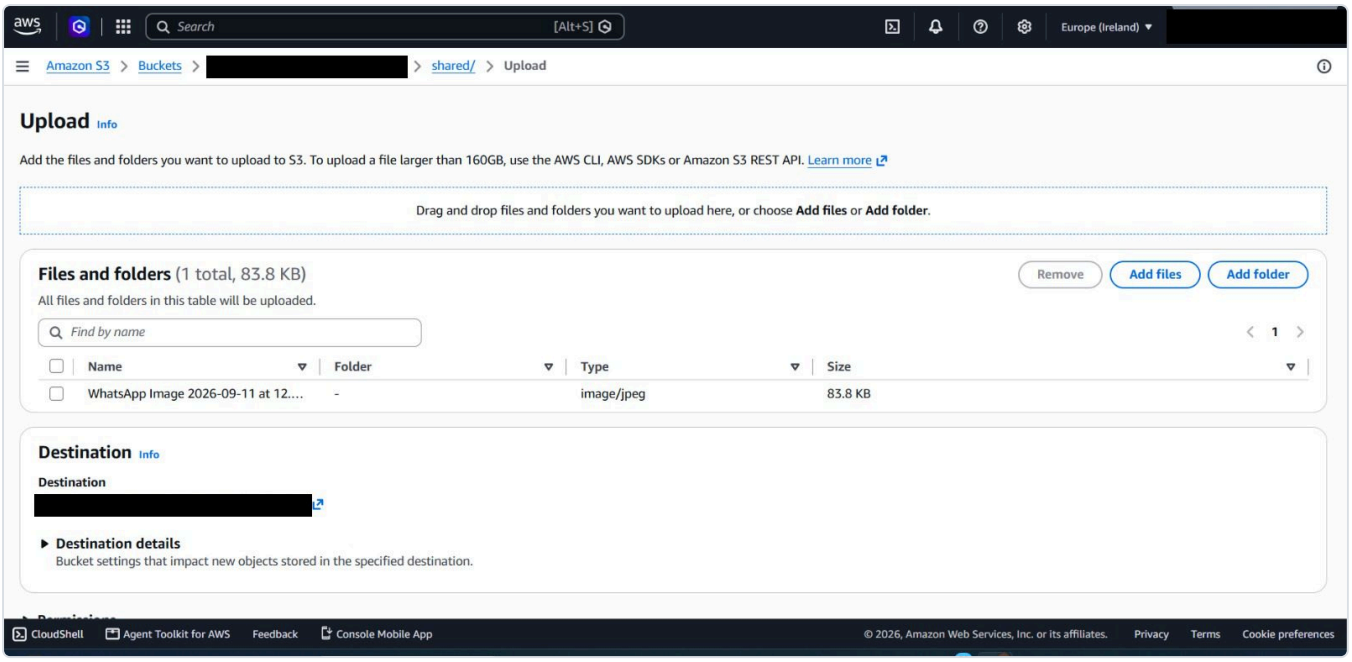


Figure 10 — Test 3: team-member-1 uploading a file to shared/.

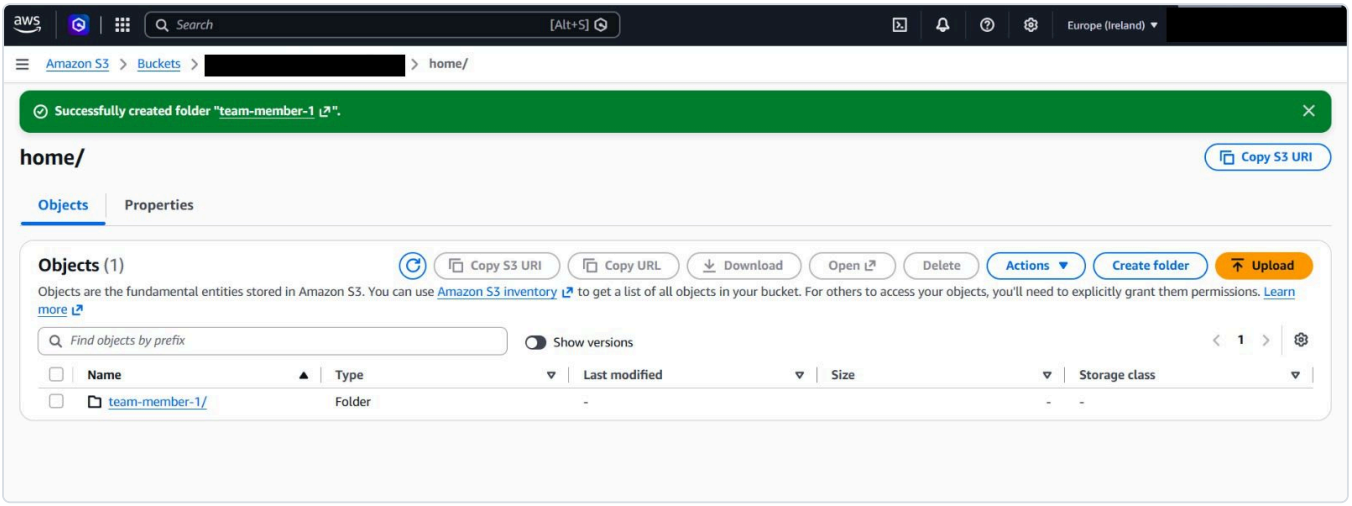


Figure 11 — Test 4: team-member-1 creates home/team-member-1/.

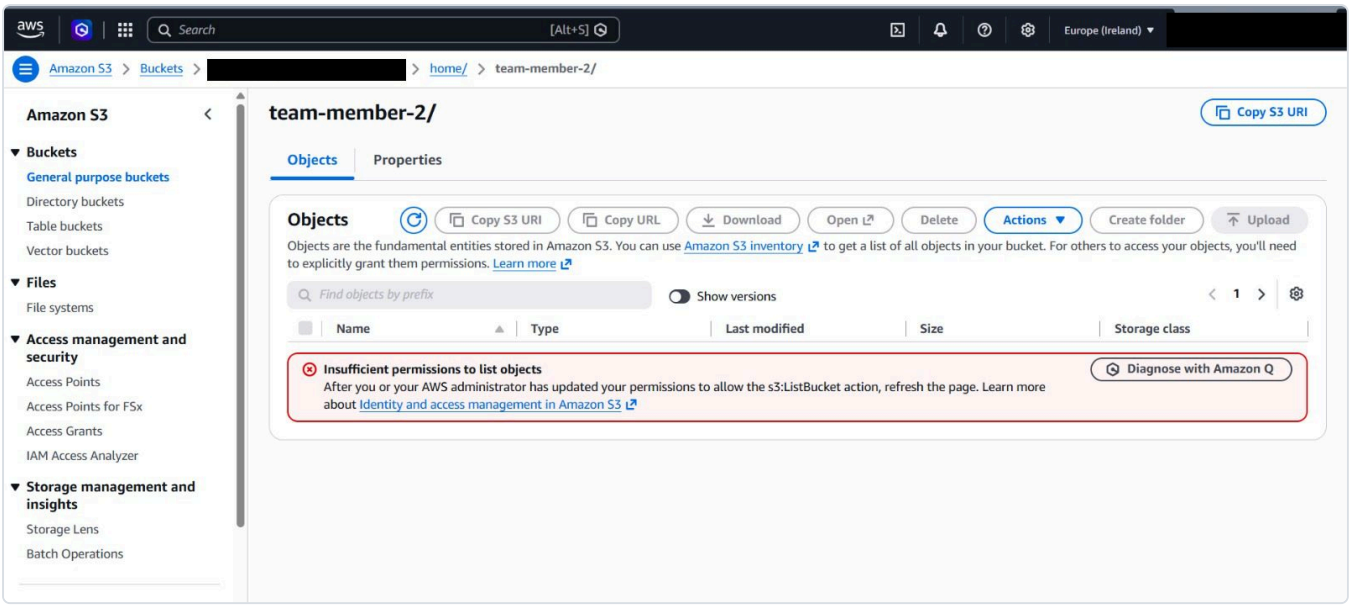


Figure 12 — Test 5: team-member-1 opens home/team-member-2/ and is refused listing.

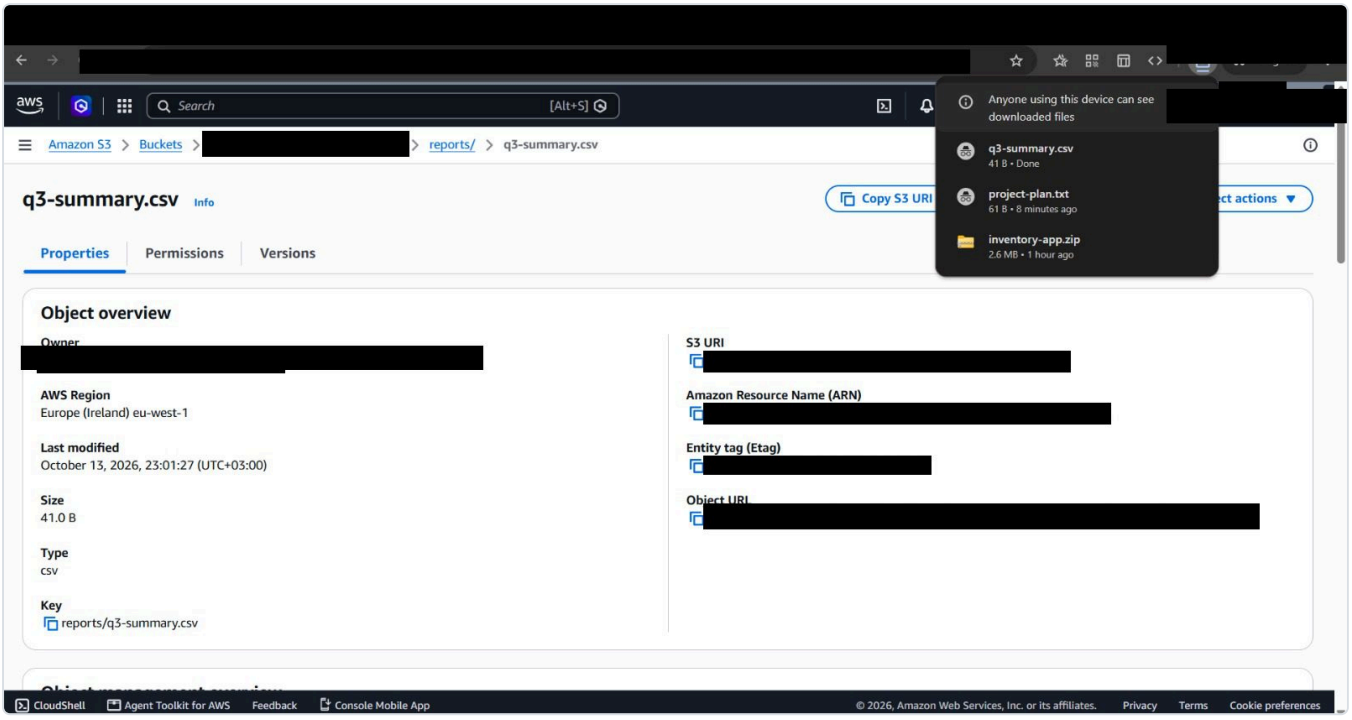


Figure 13 — Test 6: team-member-1 downloads reports/q3-summary.csv.

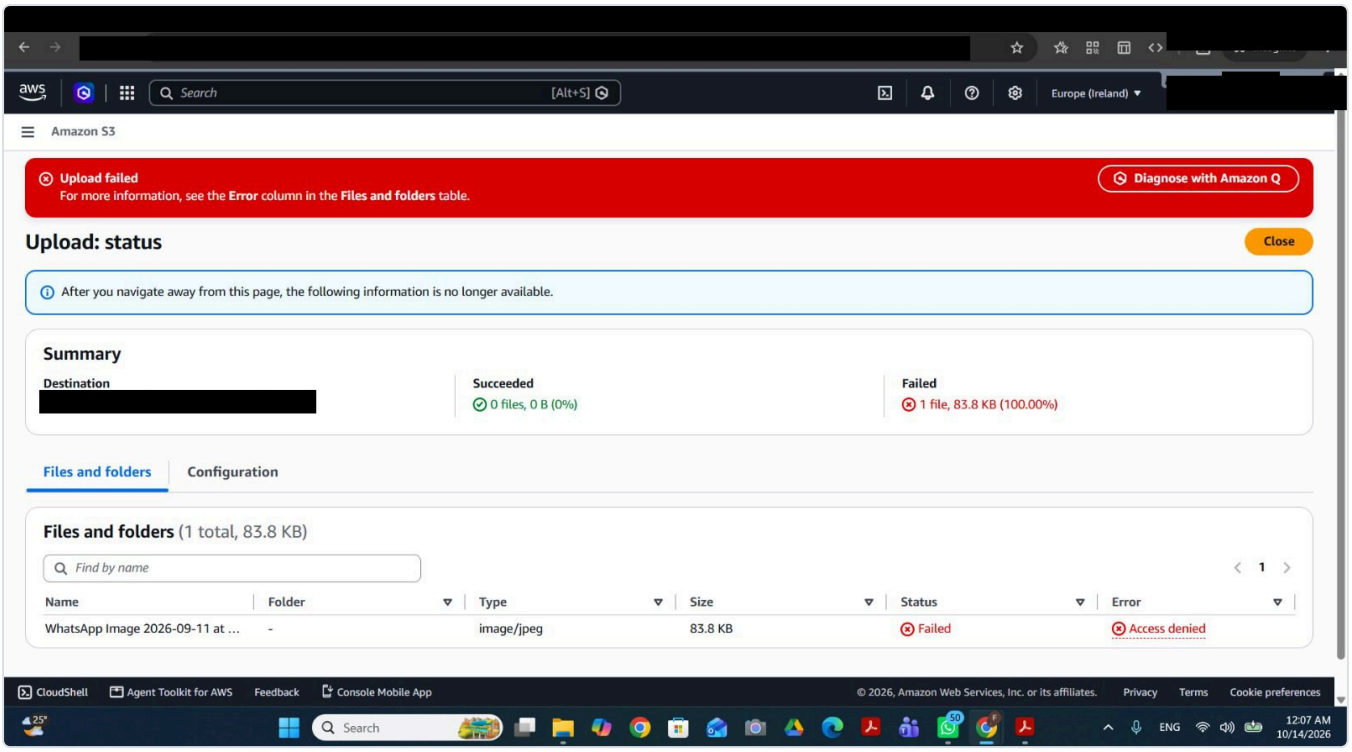


Figure 14 — Test 7: the upload to reports/ fails with Access denied.

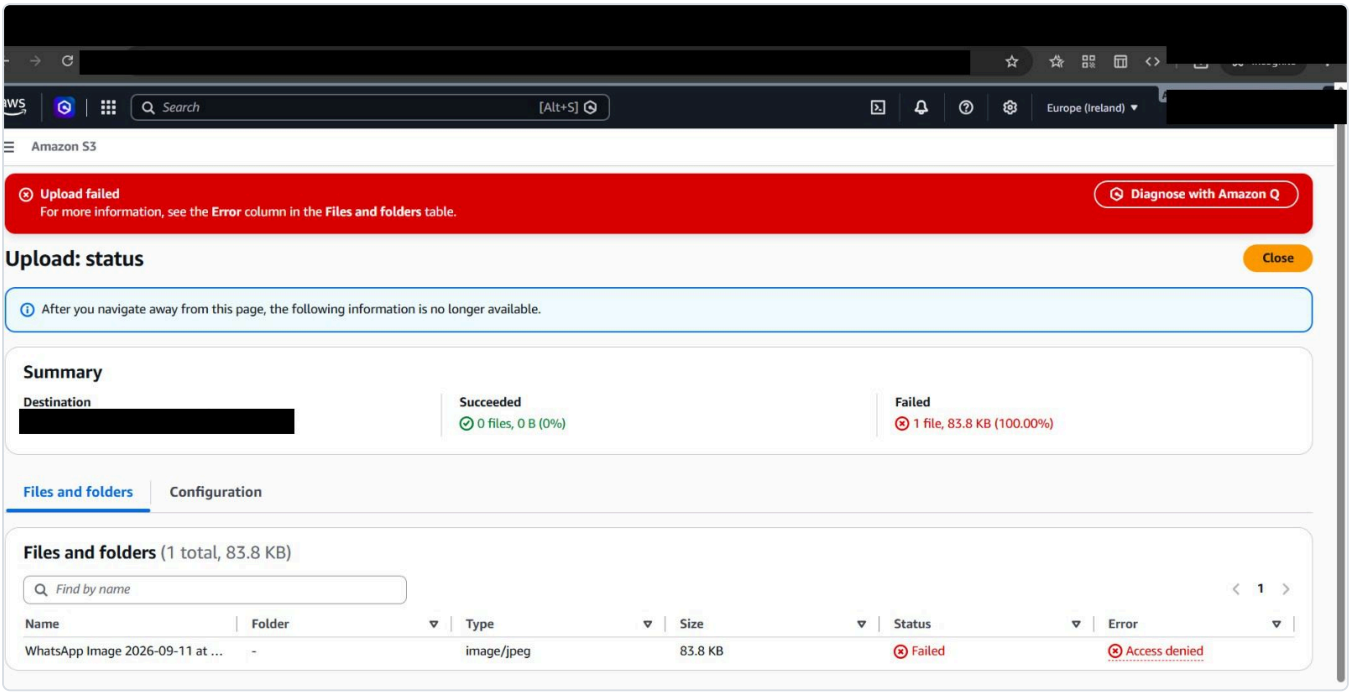


Figure 15 — Test 8: the upload to public/ fails with Access denied.

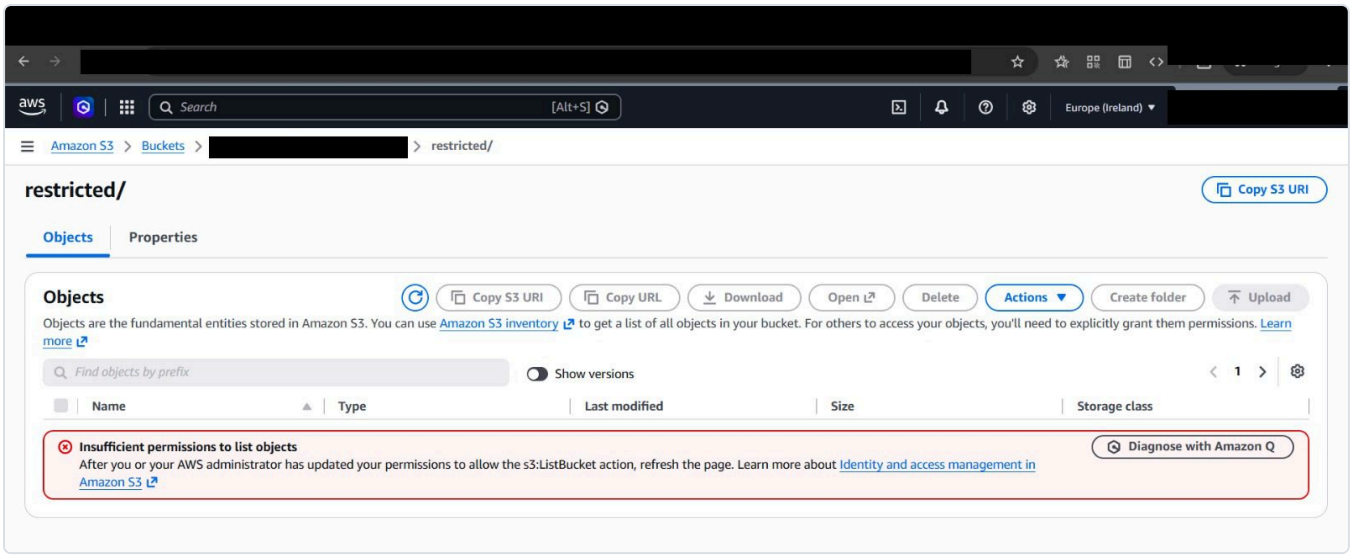


Figure 16 — Test 9: restricted/ cannot be listed.

5.2 team-member-2 (tests 4–5 swapped)

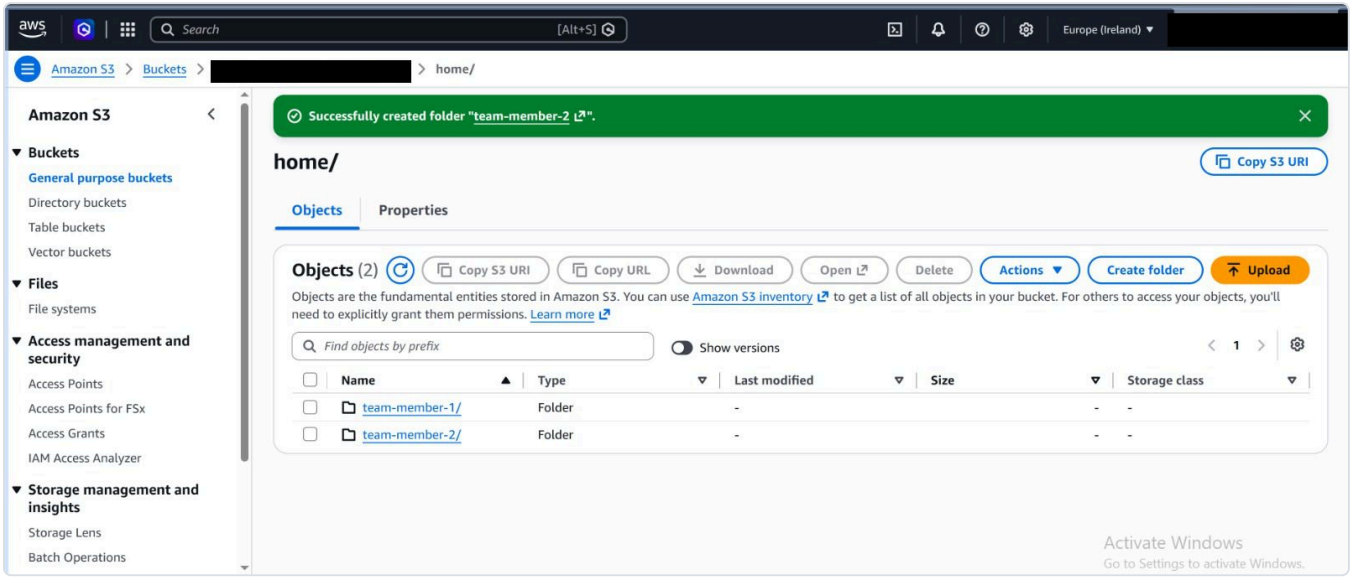


Figure 17 — team-member-2 creates home/team-member-2/. Both folder names are visible under home/.

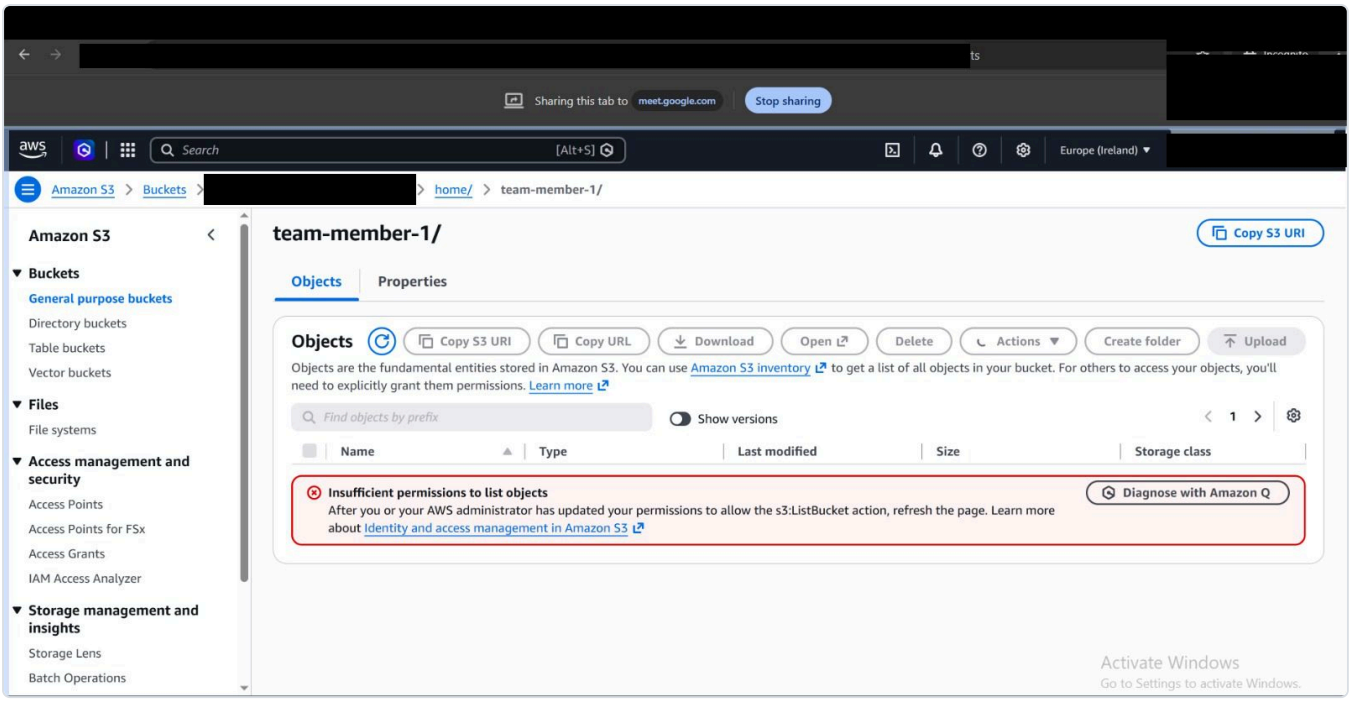


Figure 18 — team-member-2 opens home/team-member-1/ and is refused listing.

5.3 team-lead (tests 10–12)

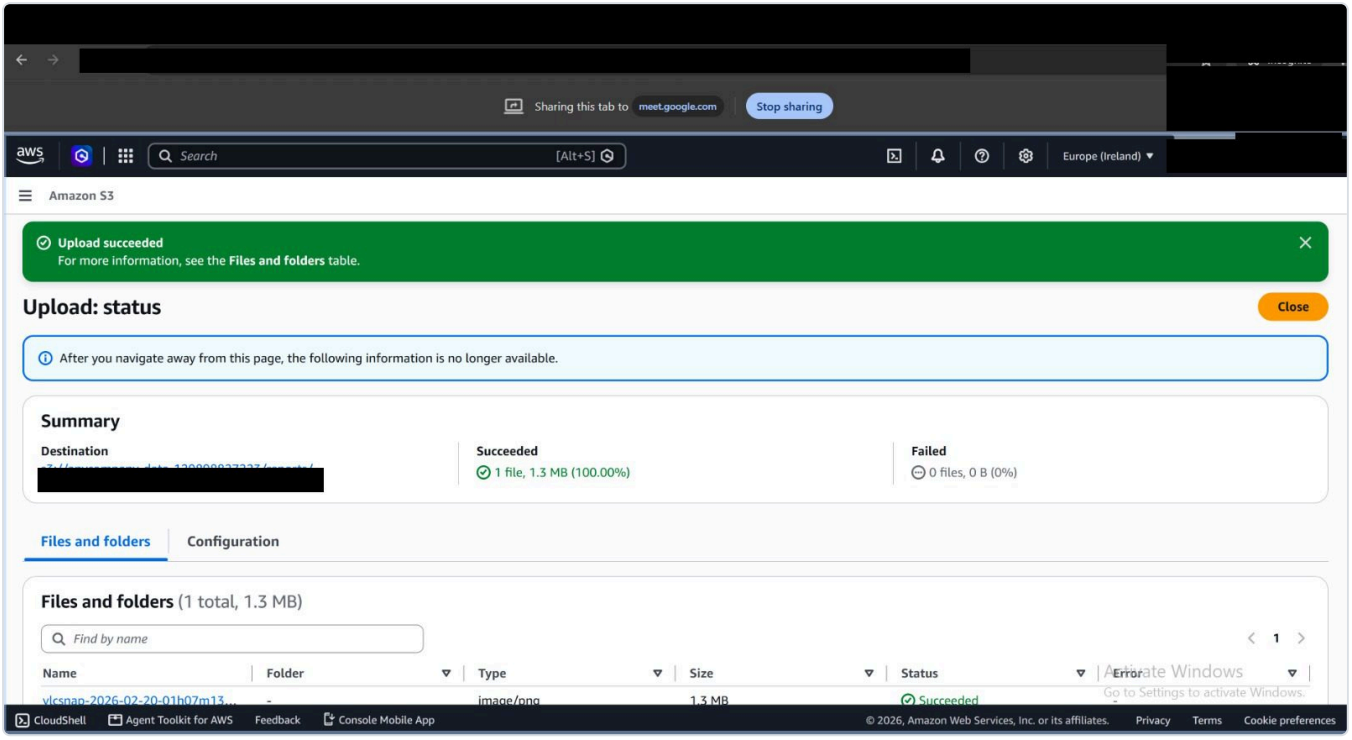


Figure 19 — Test 10: team-lead uploads a file to reports/ successfully.

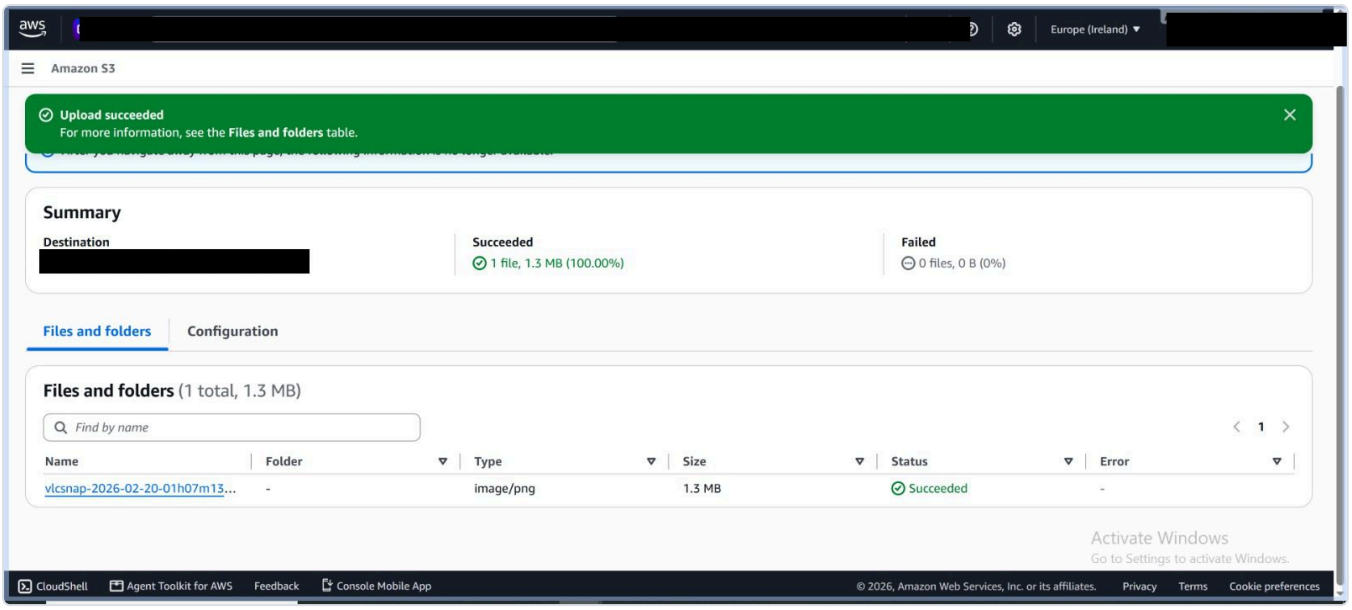


Figure 20 — Test 11: team-lead uploads a file to public/ successfully.

5.4 Anonymous visitor (tests 13–18)

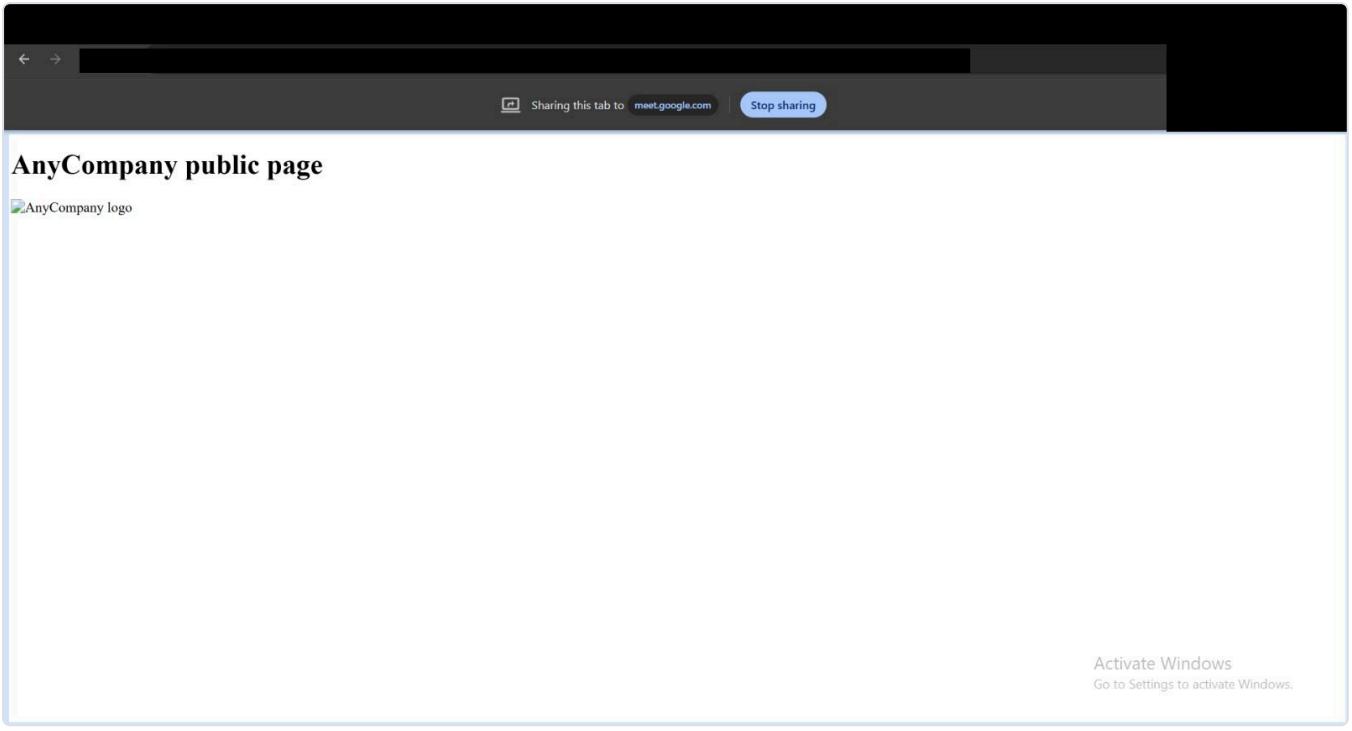


Figure 21 — Test 13: an anonymous visitor loads public/index.html over HTTPS.

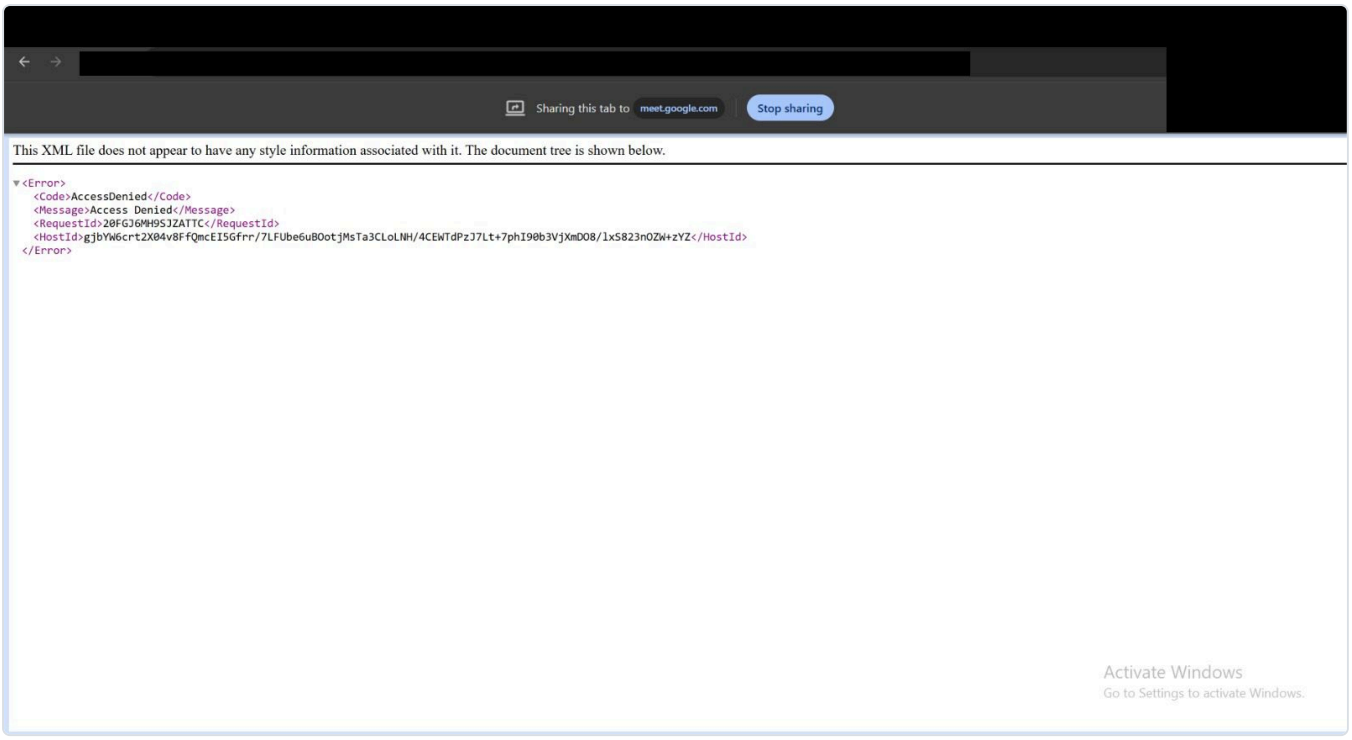


Figure 22 — Test 15: restricted/salaries.csv returns AccessDenied.

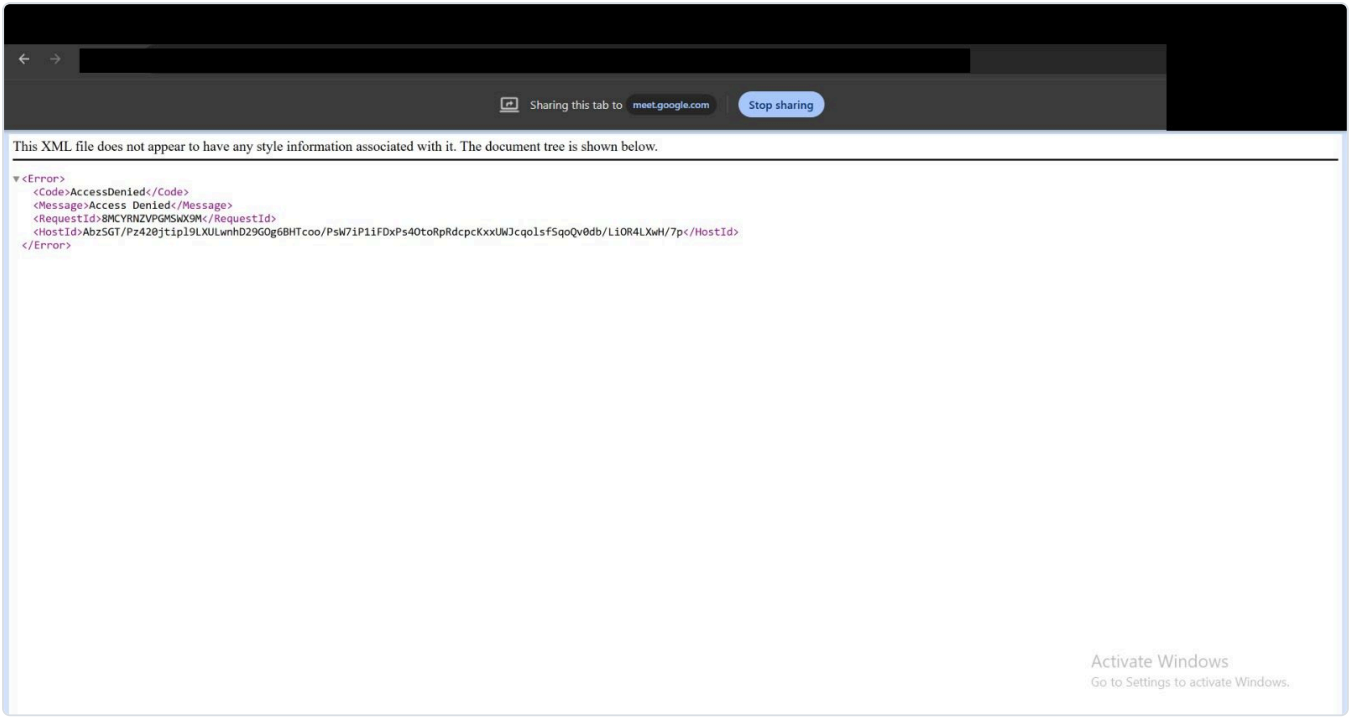


Figure 23 — Test 16: shared/project-plan.txt returns AccessDenied.

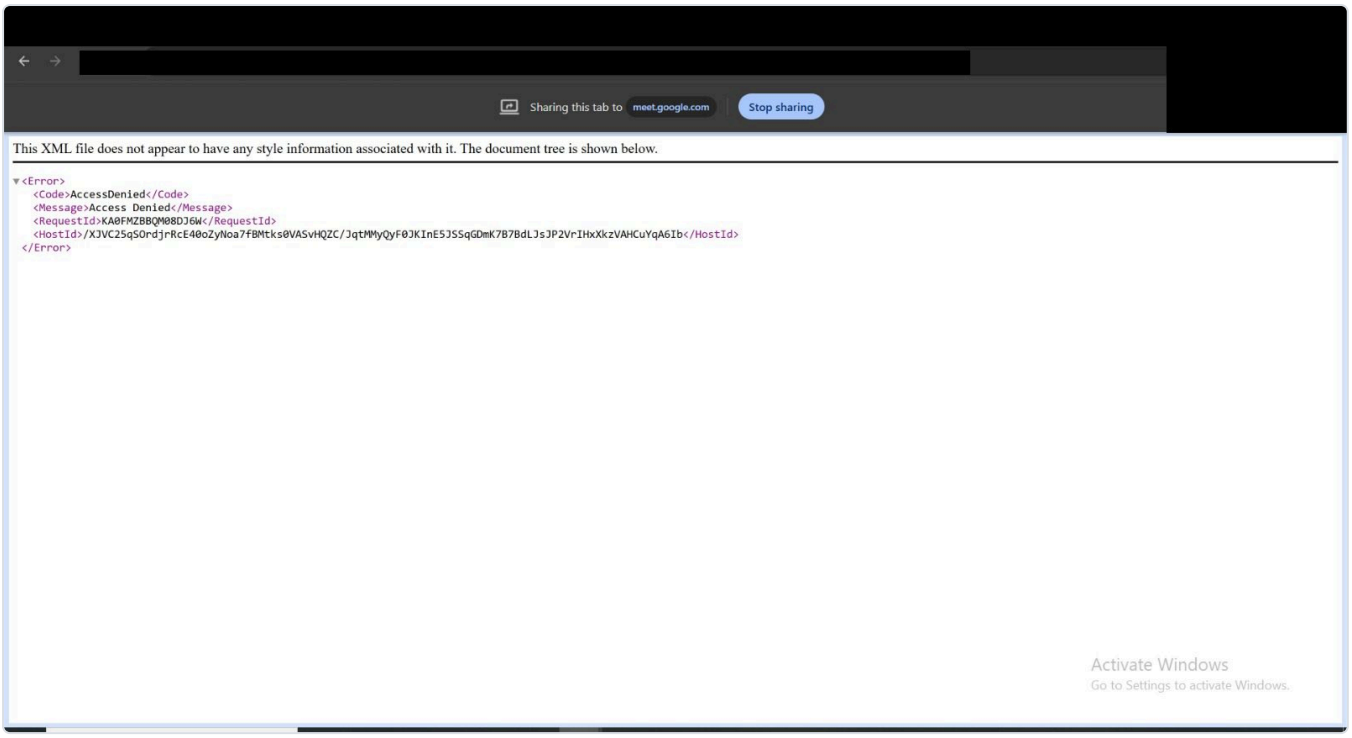


Figure 24 — Test 17: listing public/ returns AccessDenied.

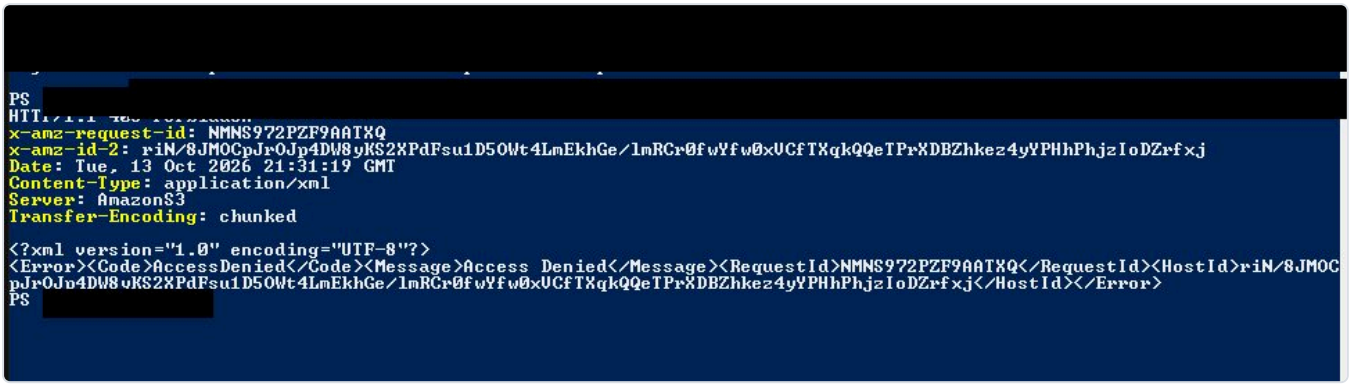


Figure 25 — Test 18: a plain-HTTP request returns HTTP/1.1 403 Forbidden with AccessDenied.

5.5 IAM Policy Simulator

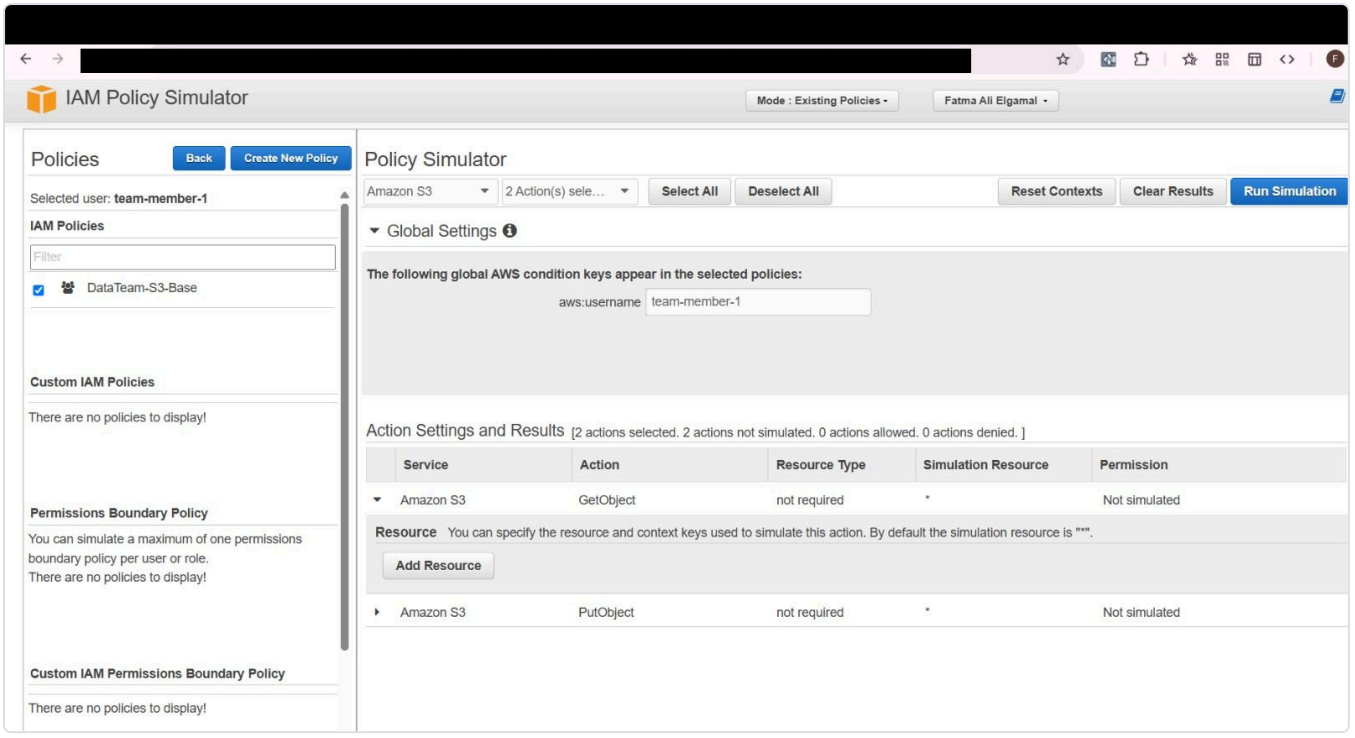


Figure 26 — Policy Simulator set up for team-member-1, with the aws:username context key set to team-member-1.

Still to capture before submission

Eight items are missing; each is a quick retest or screenshot. Run the lab's cleanup only after grading, because the grading checklist inspects the live setup.

- [ ] Task 2 checkpoint: User groups page showing DataTeam-Members with 3 users and DataTeam-Leads with 1.
- [ ] Task 3 checkpoint: team-lead's Permissions tab showing both policies marked "Attached via group".
- [ ] Test 3: the "Upload succeeded" screen for the file uploaded to shared/.
- [ ] Test 4 (required screenshot): upload a file into home/team-member-1/ and capture it.
- [ ] Test 12: sign in as team-lead, open restricted/ and capture the denial.
- [ ] Test 14: copy the exact name of the file the lead uploaded to public/, open https://anycompany-dat[redacted]23.s3.eu-west-1.amazonaws.com/public/ followed by that name in a signed-out window, and capture it loading.
- [ ] Policy Simulator: click Run Simulation for both resources and capture the denied and allowed results.
- [ ] public/logo.png: confirm it is in public/ (upload it if not). The logo shows as a broken image in Figures 7 and 21.